

```

1 import components.sequence.Sequence;
2
3 /**
4  * Implements method to smooth a {@code Sequence<Integer>}.
5  *
6  * @author Zhuangzhuang He
7  *
8  */
9 public final class SequenceSmooth2 {
10
11     /**
12      * Private constructor so this utility class cannot be instantiated.
13      */
14     private SequenceSmooth2() {
15     }
16
17     /**
18      * Smooths a given {@code Sequence<Integer>}.
19      * * @param s1
20      * the sequence to smooth
21      * @return the smoothed sequence
22      * @requires |s1| >= 1
23      * @ensures <pre>
24      * |smooth| = |s1| - 1 and
25      * for all i, j: integer, a, b: string of integer
26      * where (s1 = a * <i> * <j> * b)
27      * (there exists c, d: string of integer
28      * (|c| = |a| and
29      * smooth = c * <(i+j)/2> * d))
30      * </pre>
31      */
32     public static Sequence<Integer> smoothRecursive(Sequence<Integer> s1) {
33         assert s1 != null : "Violation of: s1 is not null";
34         assert s1.length() >= 1 : "Violation of: |s1| >= 1";
35
36         Sequence<Integer> result = s1.newInstance();
37
38         if (s1.length() > 1) {
39             int first = s1.remove(0);
40             int second = s1.entry(0);
41
42             int avg;
43             if ((first > 0 && second > 0) || (first < 0 && second < 0)) {
44                 avg = first / 2 + second / 2 + (first % 2 + second % 2) / 2;
45             } else {
46                 avg = (first + second) / 2;
47             }
48
49             result = smoothRecursive(s1);
50             result.add(0, avg);
51             s1.add(0, first);
52         }

```

```
53
54     return result;
55 }
56 public static Sequence<Integer> smoothIterative(Sequence<Integer> s1) {
57     assert s1 != null : "Violation of: s1 is not null";
58     assert s1.length() >= 1 : "Violation of: |s1| >= 1";
59
60     Sequence<Integer> result = s1.newInstance();
61     int i = 0;
62
63     while (i < s1.length() - 1) {
64         int first = s1.entry(i);
65         int second = s1.entry(i + 1);
66
67         int avg;
68         if ((first > 0 && second > 0) || (first < 0 && second < 0)) {
69             avg = first / 2 + second / 2 + (first % 2 + second % 2) / 2;
70         } else {
71             avg = (first + second) / 2;
72         }
73
74         result.add(result.length(), avg);
75         i++;
76     }
77
78     return result;
79 }
80 }
```